



ESCUELA DE
INGENIERÍA EN CIENCIAS Y SISTEMAS
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA



Fecha:

DD/MM/2026

Organización de lenguajes y compiladores 2

Escuela de Ingeniería de Ciencias Y Sistemas

**NOMBRE DEL
TUTOR**



Creación de un compilador completo

Agenda

Recordatorios:

- Foro de esta semana
- Horarios de calificación
- Tarea 2
- Próximas clases



Recursos:

- [RISC-V Online](#)
- [Crafting Interpreters](#)



Definiciones

Environment

Implementa la tabla de símbolos. En cada push o pop se crea esta estructura de datos para guardar todas las variables, funciones, arreglos, clases, instancias y métodos, es decir todos los identificadores del lenguaje IMPLEMENTADO.

```
Env.push(superior);  
superior = new Env(tope);
```

Podríamos borrar el env que sale (pop) e ir agregando los símbolos en el reporte.

Nombre	Tipo	Alcance
valor1	integer	global
cadena	string	global
i	integer	func_suma()
j	integer	func_suma()

Memoria

En la arquitectura ARM64 solamente un espacio virtual de memoria lineal, la subdivisión es gestionada por nosotros en tiempo de ejecución.

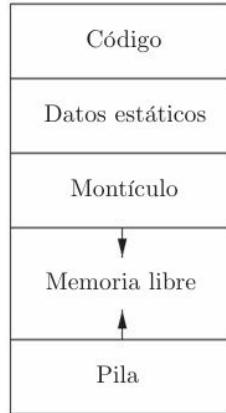


Figura 7.1: Subdivisión típica de la memoria en tiempo de ejecución, en áreas de código y de datos

Registros de activación

La asignación de espacio en la pila en tiempo de ejecución generalmente se administra de casi igual en la mayoría de compiladores.

Por lo general, las llamadas a los procedimientos y los retornos de los mismos se manejan mediante una pila en tiempo de ejecución, conocida como pila de control. Cada activación en vivo tiene un registro de activación (algunas veces se le conoce como marco) en la pila de control

Parámetros actuales
Valores devueltos
Enlace de control
Enlace de acceso
Estado de la máquina guardado
Datos locales
Temporales

Descriptores de registro y direcciones

1. Para cada registro disponible, un descriptor de registro lleva la cuenta de los nombres de las variables cuyo valor actual se encuentra en ese registro.
2. Para cada variable del programa, un descriptor de acceso lleva la cuenta de la ubicación o ubicaciones en las que puede encontrarse el valor actual de esa variable. La ubicación podría ser un registro, una dirección de memoria, una ubicación en la pila o algún conjunto de más de uno de éstos.

Descriptores de registro y direcciones

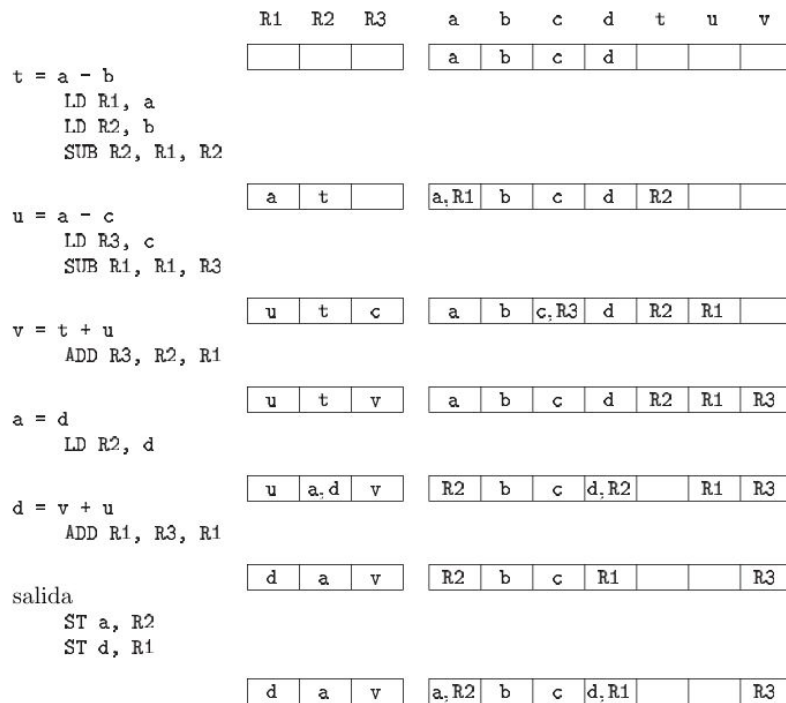


Figura 8.16: Instrucciones generadas y los cambios en los **descriptores** de registro y de dirección

Generador código intermedio

Implementa la estructura de datos para las instrucciones en representación intermedia (Cuadрупlos, tripletas o tripletas indirectas).

Tiene los métodos para agregar todas las diferentes instrucciones del RI utilizado (C3D, Aarch64 y más) agregando cada instrucción en una lista de la estructura de datos escogida.

Output->Instrucciones ARM en cuadрупlos. Cuenta con un método para generar el reporte en un archivo .s.

	<i>op</i>	<i>arg₁</i>	<i>arg₂</i>	<i>resultado</i>
0	enos	c		t ₁
1	*	b	t ₁	t ₂
2	enos	c		t ₃
3	*	b	t ₃	t ₄
4	+	t ₂	t ₄	t ₅
5	=	t ₅		a
		...		

(b) Cuádruplos

Estructura
en código

```
class Instruction {  
  
    public $instr;  
    public $op1;  
    public $op2;  
    public $op3;  
  
    public function __construct($instr, $op1=null, $op2=null, $op3=null) {  
        $this->instr = $instr;  
        $this->op1 = $op1;  
        $this->op2 = $op2;  
        $this->op3 = $op3;  
    }  
  
    public function __toString() {  
  
        $ops = [];  
  
        if ($this->op1 != null) $ops[] = $this->op1;  
        if ($this->op2 != null) $ops[] = $this->op2;  
        if ($this->op3 != null) $ops[] = $this->op3;  
  
        if (count($ops) == 0) return $this->instr;  
  
        return $this->instr . " " . implode(" ", $ops);  
    }  
}
```



Objetivo

¿Qué debe hacer nuestro compilador?

Código entrada

```
private function initRegisters() {
    $this->specialRegisters['xzn'] = new RegisterDescriptor('xzn');
    $this->specialRegisters['sp'] = new RegisterDescriptor('sp');
    $this->specialRegisters['fp'] = new RegisterDescriptor('fp');
    $this->specialRegisters['lr'] = new RegisterDescriptor('lr');
    $this->specialRegisters['x30'] = new RegisterDescriptor('x30');
    $this->specialRegisters['x29'] = new RegisterDescriptor('x29');

    for ($i = 0; $i <= 7; $i++) {
        $this->registers['x' . $i] = new RegisterDescriptor('x' . $i);
    }

    for ($i = 9; $i <= 15; $i++) {
        $this->temporaryRegisters['x' . $i] = new RegisterDescriptor('x' . $i);
    }

    // x19-x28 son registros callee-saved, pero los incluimos como temporales
    for ($i = 19; $i <= 28; $i++) {
        $this->registers['x' . $i] = new RegisterDescriptor('x' . $i);
    }

    $this->registers['x16'] = new RegisterDescriptor('x16');
    $this->registers['x17'] = new RegisterDescriptor('x17');
    $this->registers['x18'] = new RegisterDescriptor('x18');
}
```

Representación intermedia en cuádruplos

Op1	Op2	result	tipo
---	---	---	---
---	---	---	---
---	---	---	---

Código ensamblador

```
.global _start
.section .bss
buffer: .skip 32
.section .text
_start:
    // Cargando entero: 5
    mov x9, #5
    // Cargando entero: 3
    mov x10, #3
    // Visitando expresión relacional: <
    cmp x9, x10
    cset x11, lt

.section .rodata
newline: .asciz "\n"
```



Paso de argumentos

```
]
=
.
```

Video Explicativo

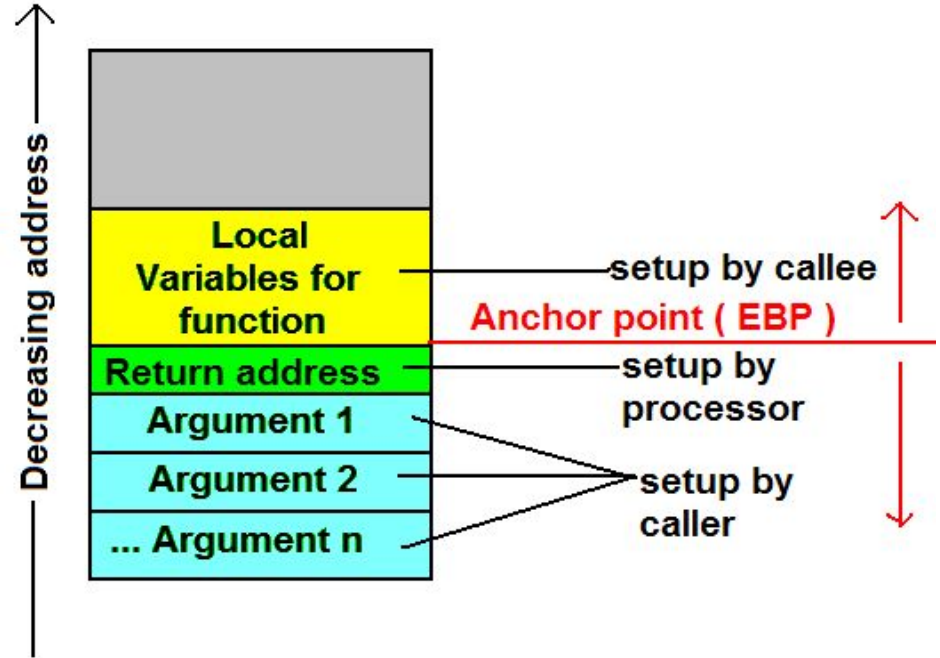
Paso de argumentos

Normalmente el paso de argumentos se hace desde los registros específicos para esa función, en el caso de RISC-V son a0-a9. Sin embargo cuando son varios argumentos es necesario hacer uso del stack como hemos visto en el video.

El stack funciona como intermediario entre el caller y el callee, sin embargo es necesario tener una estructura y diseño para que no hayan errores futuros.

Call frame

El call frame es ese diseño y estructura que necesitamos a la hora de hacer una llamada a una función.



```
fun first() {  
  var a = 1;  
  second();  
  var b = 2;  
}
```

```
fun second() {  
  var c = 3;  
  var d = 4;  
}
```

```
first();
```

call first()

var a = 1;

call second()

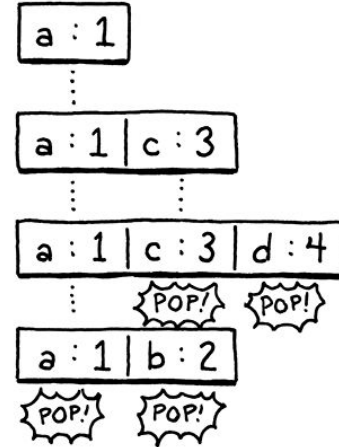
var c = 3;

var d = 4;

return from second()

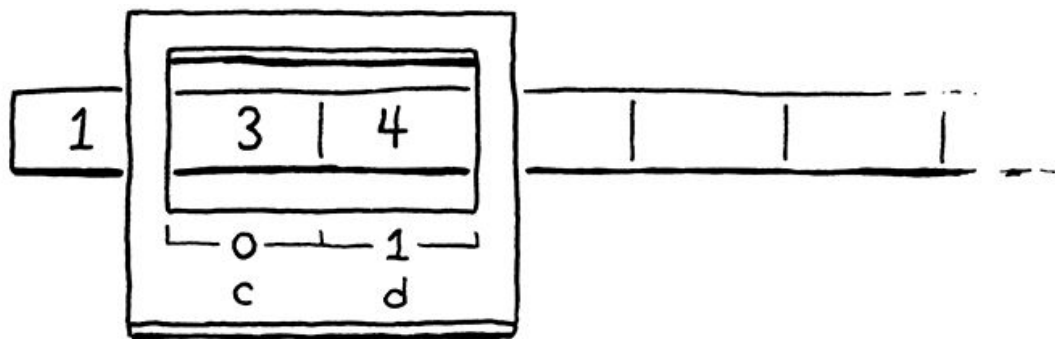
var b = 2;

return from first()

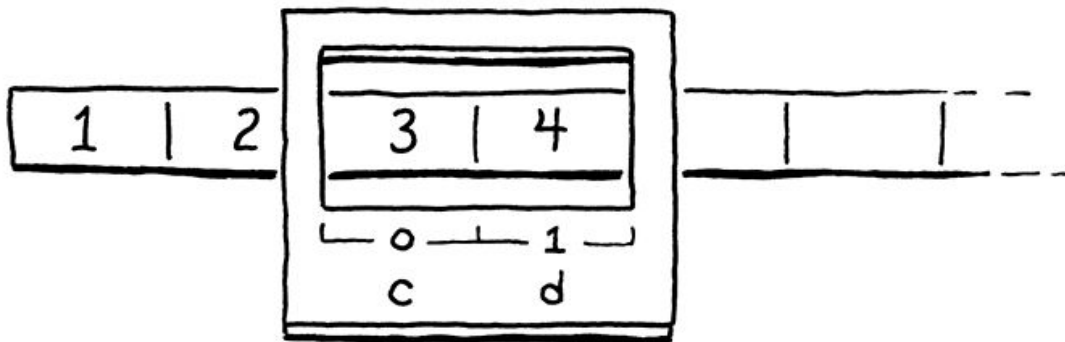


Ejemplo de Crafting Interpreters

first call to second():



second call to second():



Parameters and return
Call stack
History
Stack point...

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34

```

.data
newline: .asciz "\n"

.text
.globl main

main:
    # Prologue
    addi sp, sp, -16
    sw   ra, 12(sp)
    sw   s0, 8(sp)
    mv   s0, sp

    # Push arguments: a=1, b=2, c=3
    addi sp, sp, -4
    li   t0, 1
    sw   t0, 0(sp)

    addi sp, sp, -4
    li   t0, 2
    sw   t0, 0(sp)

    addi sp, sp, -4
    li   t0, 3
    sw   t0, 0(sp)

    call sum3

    # Pop arguments (3 x 4 bytes)
    addi sp, sp, 12

    # Epilogue
    lw   ra, 12(sp)
    lw   s0, 8(sp)

```

PC 00000000
Registers B W L
RA 0000 0000
SP 0000 0000
GP 0000 0000
TP 0000 0000
T0 0000 0000
T1 0000 0000
T2 0000 0000
S0 0000 0000
S1 0000 0000
A0 0000 0000
A1 0000 0000
A2 0000 0000
A3 0000 0000
A4 0000 0000
A5 0000 0000
A6 0000 0000
A7 0000 0000
S2 0000 0000
S3 0000 0000
S4 0000 0000
S5 0000 0000
S6 0000 0000
S7 0000 0000
S8 0000 0000

Address 10010000

Tr
00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F

10010000 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
10010010 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
10010020 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
10010030 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
10010040 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
10010050 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
10010060 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
10010070 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
10010080 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
10010090 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
100100A0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
100100B0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
100100C0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
100100D0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
100100E0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
100100F0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

Build
Testcases

Ejemplo en tiempo real con registros